Docker Guide: Dockerfile Basics and Commands (Part 1 and Part 2)

Project folder: `c:\Users\stej\OneDrive - Emerson\Desktop\Docker`

The Dockerfile from the basics

A Dockerfile is a text file of instructions that describes the environment your app needs. Docker uses it to build a package that runs the same way anywhere.

Term	Meaning	Analogy
Dockerfile	Instructions	Recipe
Image	Packaged, read-only result	Frozen meal
Container	A running copy of an image	Meal being served

Flow: **Dockerfile** → **(** `docker build` **)** → **Image** → **(** `docker run` **)** → **Container**

Our Dockerfile, line by line

Line	Meaning
`FROM python:3.13-slim`	Start from a small Linux image with Python 3.13 installed. `python` is the image name, `3.13-slim` is the tag.
`WORKDIR /app`	Create /app inside the image and make it the current folder.
`COPY requirements.txt .`	Copy the file from your PC into the image. Format: COPY <from PC> <to image>. The `.` is the current image folder (/app).
`RUN pip install --no-cache-dir -r requirements.txt`	Run a command at build time. Installs Flask into the image.
`COPY src/app.py .`	Copy your code to /app/app.py.
`EXPOSE 5000`	Documents that the app listens on port 5000. It does not publish the port; `-p` does that at run time.
`CMD ["python", "app.py"]`	Command run when a container starts (same as typing python app.py).

Key ideas

- **Layers and caching:** each instruction creates a layer, and Docker reuses unchanged layers. requirements.txt is installed before the code is copied, so editing app.py skips the slow install.
- **Build time vs run time:** FROM, WORKDIR, COPY and RUN happen at `docker build`. CMD happens at `docker run`.
- **Image vs container:** one image can start many containers. Changing a file on your PC does not change an existing image; rebuild it.

Common instructions

Instruction	Purpose
FROM	base image
WORKDIR	set the working folder
COPY	copy files in
RUN	run a command at build time
ENV	set an environment variable
EXPOSE	document a port
CMD	default command at start

Part 1: Run the Dockerfile alone

#	Command	What it does	What you'll see
1	`cd "c:\Users\stej\OneDrive - Emerson\Desktop\Docker"`	Moves into the project folder.	No output
2	`docker compose down`	Clears earlier compose containers so the ports are free.	Removed lines
3	`docker build -t tejas-image .`	Builds the image from the Dockerfile and names it tejas-image.	Build steps ending with naming to ...tejas-image:latest
4	`docker images tejas-image`	Confirms the image exists.	A row for tejas-image
5	`docker run -d --name tejas-container -p 5000:5000 tejas-image`	Starts a container named tejas-container in the background. PC port 5000 goes to container port 5000.	A long container ID
6	`docker ps`	Shows it's running.	tejas-container with status Up
7	`curl.exe localhost:5000`	Tests the app (or open http://localhost:5000).	Hello from Flask in Docker!
8	`docker logs tejas-container`	Shows the app's output.	Flask startup lines and your request
9	`docker rm -f tejas-container`	Stops and deletes the container.	tejas-container

Part 2: Run the compose file

Finish Part 1 step 9 first so port 5000 is free.

#	Command	What it does	What you'll see
10	`docker compose config`	Checks the compose file is valid.	The file printed back, no errors
11	`docker compose up -d --build`	Builds the web image and starts both containers in the background.	hello-compose Started, hello-nginx Started
12	`docker compose ps`	Shows the compose containers.	Both Up, ports 8001->5000 and 8002->80
13	`curl.exe localhost:8001`	Tests Flask through compose (or open http://localhost:8001).	Hello from Flask in Docker!
14	`curl.exe -I localhost:8002`	Tests nginx (or open http://localhost:8002).	HTTP/1.1 200 OK
15	`docker compose logs`	Shows output of both containers.	Flask startup lines and nginx request logs
16	`docker exec hello-compose ls /app`	Looks inside the Flask container's working directory.	app.py and requirements.txt
17	`docker compose down`	Stops and removes the containers and network.	Removed lines

Clean up images (after step 17)

```
docker rmi tejas-image hello-python
```

Notes

- Compose builds its own image called hello-python, with containers hello-compose and hello-nginx. It does not use tejas-image unless you change `image: hello-python` to `image: tejas-image` in docker-compose.yaml.
- "error during connect" means Docker Desktop isn't running.
- "port is already allocated" means another container is using that port. Check `docker ps`.